

ANHANGUERA-LONDRINA/PR

ENGENHARIA DE SOFTWARE

PORTFÓLIO - RELATÓRIO DE AULA PRÁTICA:

NOME DA DISCIPLINA: PROGRAMAÇÃO WEB

Londrina/PR
2026

PORTFÓLIO – RELATÓRIO DE AULA PRÁTICA

NOME DA DISCIPLINA: PROGRAMAÇÃO WEB

Trabalho de portfólio apresentado como requisito parcial para a obtenção de pontos para a média semestral.

Orientador: Tutor

SUMÁRIO

1. INTRODUÇÃO..... 4
2. DESENVOLVIMENTO..... 4
3. CONCLUSÃO..... 6

1. INTRODUÇÃO

Neste trabalho, desenvolvi uma aplicação web dinâmica para consulta de signos zodiacais. A solução foi construída utilizando PHP no back-end e um arquivo XML como base de dados.

A ideia principal do projeto foi trabalhar com leitura de dados externos e lógica de programação para tratar datas. Além disso, utilizei o Bootstrap para garantir uma interface mais organizada e responsiva.

2. DESENVOLVIMENTO

2.1 Estrutura do Projeto

Seguindo o roteiro proposto, organizei o projeto dentro da pasta `Project` no diretório `htdocs` do XAMPP, com a seguinte estrutura:

- `layouts/header.php`: responsável pelo cabeçalho reutilizável e importação de estilos
- `index.php`: página principal com o formulário
- `show_zodiac_sign.php`: responsável por processar os dados e exibir o resultado
- `signos.xml`: arquivo que contém os dados dos signos

2.2 Implementação do Código

2.2.1 Base de Dados (`signos.xml`)

O arquivo XML funciona como a base de dados da aplicação, contendo os intervalos de datas e as descrições dos signos:

```
<?xml version="1.0" encoding="UTF-8"?>
<signos>
  <signo>
    <dataInicio>21/03</dataInicio>
    <dataFim>20/04</dataFim>
    <signoNome>Áries</signoNome>
    <descricao>Signo de fogo, regido por Marte. Simboliza o início e a
coragem.</descricao>
  </signo>
  <signo>
```

```
        <dataInicio>21/04</dataInicio>
        <dataFim>20/05</dataFim>
        <signoNome>Touro</signoNome>
        <descricao>Signo de terra, regido por Vênus. Foca em estabilidade e prazeres sensoriais.</descricao>
    </signo>
</signos>
```

2.2.2 Interface e Formulário (`header.php` e `index.php`)

Para manter o código organizado e evitar repetição, separei o cabeçalho em um arquivo próprio. O formulário utiliza o método **POST** para enviar a data de nascimento para a página de resultado.

```
<?php include('layouts/header.php'); ?>
<div class="container mt-5">
    <div class="card shadow p-4">
        <h2>Descubra seu Signo</h2>
        <form method="POST" action="show_zodiac_sign.php">
            <div class="mb-3">
                <label class="form-label">Data de Nascimento</label>
                <input type="date" class="form-control" name="data_nascimento" required>
            </div>
            <button type="submit" class="btn btn-primary w-100">Consultar</button>
        </form>
    </div>
</div>
```

2.2.3 Lógica de Negócio (`show_zodiac_sign.php`)

Aqui fica a parte principal da aplicação, onde o **PHP** processa a data informada e compara com os dados do **XML** para identificar o signo correto.

```
<?php
include('layouts/header.php');

$data_nascimento = $_POST['data_nascimento'];

$data_usuario = new DateTime($data_nascimento);
$dia_mes_usuario = $data_usuario->format('d/m');

$xml = simplexml_load_file("signos.xml");
```

```
echo "<div class='container mt-5 text-center'>";
foreach ($xml->signo as $item) {
    if ($dia_mes_usuario >= (string)$item->dataInicio && $dia_mes_usuario <= (string)$item->dataFim) {
        echo "<h1 class='text-primary'>{$item->signoNome}</h1>";
        echo "<p class='lead'>{$item->descricao}</p>";
        break;
    }
}
echo "<a href='index.php' class='btn btn-secondary'>Voltar</a>";
echo "</div>";
?>
```

2.3 Análise da Lógica

O principal ponto desse projeto foi entender que os signos dependem apenas do dia e do mês, não do ano. Por isso, converti a data informada pelo usuário para o formato `d/m`, facilitando a comparação com o XML.

O uso do `simplexml_load_file` ajudou bastante, porque permite trabalhar com o XML de forma simples, como se fosse uma estrutura de objetos, deixando o código mais limpo e fácil de entender.

3. CONCLUSÃO

Esse projeto foi importante para reforçar a integração entre front-end e back-end. Também ajudou a entender melhor como organizar um sistema separando interface, lógica e dados.

No geral, o resultado foi uma aplicação funcional, organizada e fácil de manter, atendendo ao que foi proposto na atividade.